AVR MICROCONTROLLER: history and features
8086 Microprocessors
Introduction to Assembly Language

Objectives:

- What Is an AVR Microcontroller?
- History of AVR
- Main Features of AVR
- Applications of AVR Microcontrollers
- 8086 Microprocessors
- Introduction to Assembly Language
- Pipelining and Registers
- Pushing and Popping Operations (Stack)
- Flag Registers and bit fields

The AVR Microcontroller: History and Features

What Is an AVR Microcontroller?

An AVR microcontroller is a small computer built into a single integrated circuit (IC). It is designed to control electronic devices and perform specific tasks.

An AVR microcontroller typically contains a CPU, memory, input/output (I/O) ports, timers, and other built-in peripherals on one chip.

For example, an AVR microcontroller can read the temperature from a sensor and turn on a fan when the temperature becomes too high.

History of AVR

- **1996**: AVR was developed by two Norwegian engineers, Alf-Egil Bogen and Vegard Wollan, while they were students at the Norwegian Institute of Technology.
- **Late 1990s**: Atmel introduced AVR microcontrollers commercially. They became popular because they were relatively easy to program and suitable for embedded systems.
- **2000s**: AVR microcontrollers became widely used in educational projects, robotics, and electronic control systems.
- **Today**: AVR microcontrollers are part of Microchip Technology's product portfolio. They are still used in embedded systems and education, including many Arduino boards.

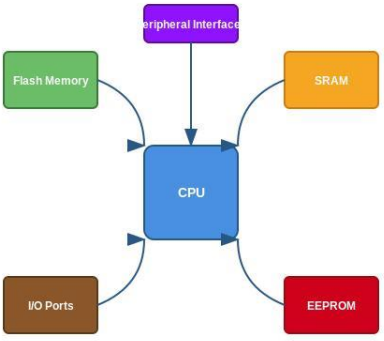
The ATmega328P, used in the classic Arduino Uno, is one of the best-known AVR microcontrollers.

Main Features of AVR



1. 8-bit RISC Architecture

Many popular AVR microcontrollers use an 8-bit CPU and a RISC (Reduced Instruction Set Computer) architecture. They are designed to execute instructions efficiently.

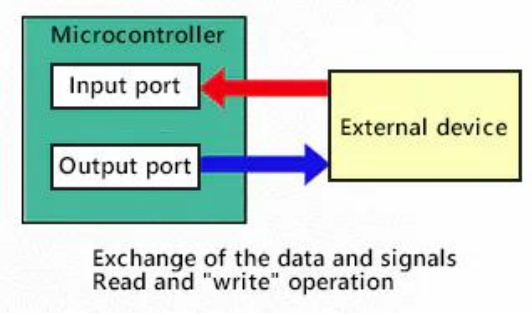


2. Built-in Memory

- Flash: Stores the program.
- SRAM: Temporarily stores data while the program runs.
- EEPROM: Stores data that must remain after power is switched off.
- Memory sizes depend on the AVR model.

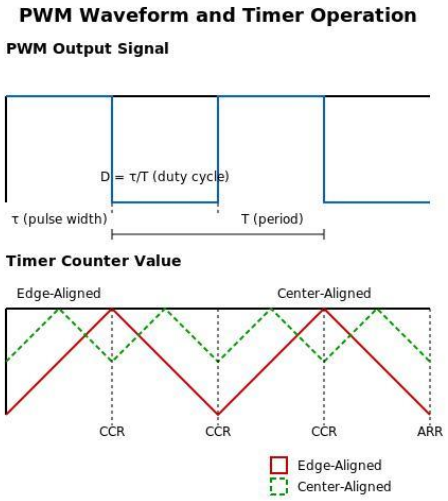
3. Input/Output (I/O) Ports

I/O pins allow the microcontroller to communicate with external devices, such as LEDs, switches, sensors, and displays.



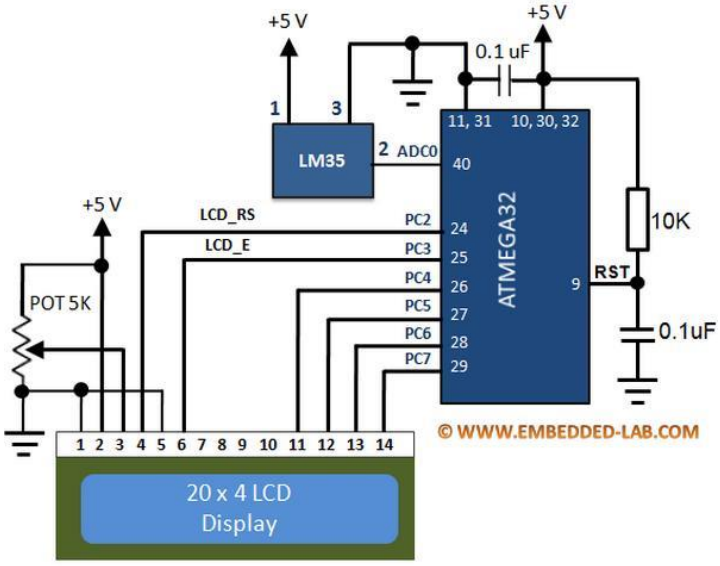
4. Timers and Counters

These helps measure time, count events, generate delays, and produce PWM signals to control devices such as motors.



5. Analog-to-Digital Converter (ADC)

Many AVR models include an ADC, which converts analog signals from sensors into digital values that the CPU can process.



6. Low Power Consumption and Easy Programming

Many AVR devices are designed for low-power operation. They can be programmed using C, assembly language, or development environments such as the Arduino IDE.

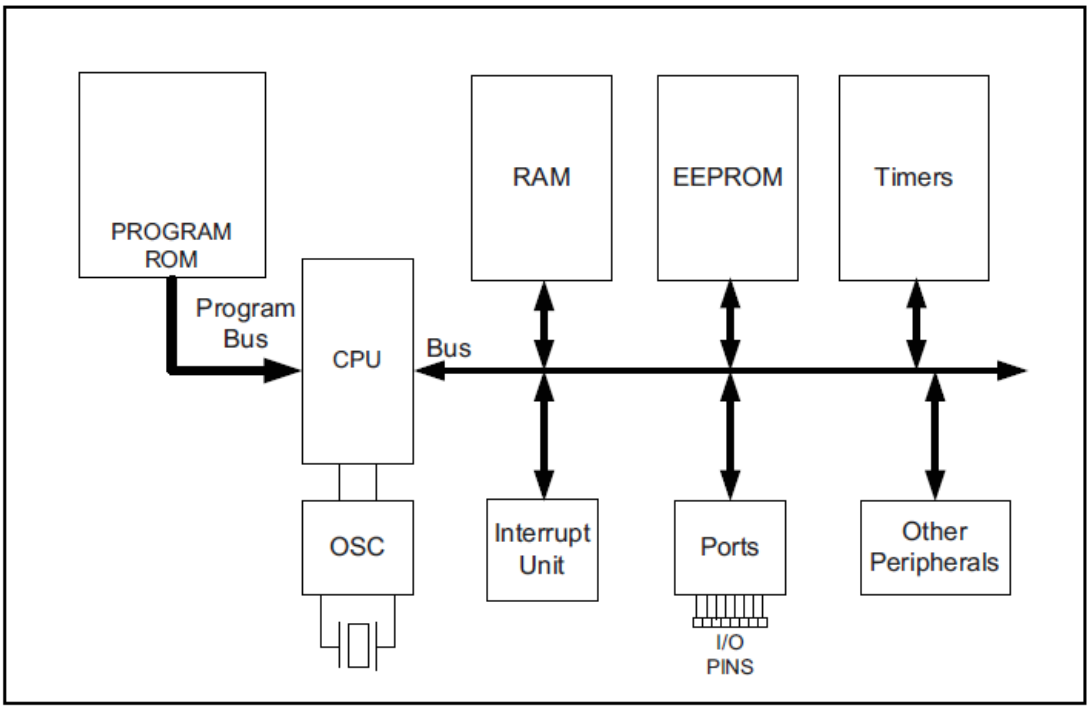


Applications of AVR Microcontrollers

AVR microcontrollers are used in many embedded systems, including:

- Home automation: Controlling lights and fans.
- Robotics: Controlling motors and reading sensors.
- Automotive electronics: Monitoring and controlling certain vehicle functions.
- Measurement systems: Reading temperature, light, and other sensor values.
- Educational projects: Learning programming, electronics, and microcontroller operation.

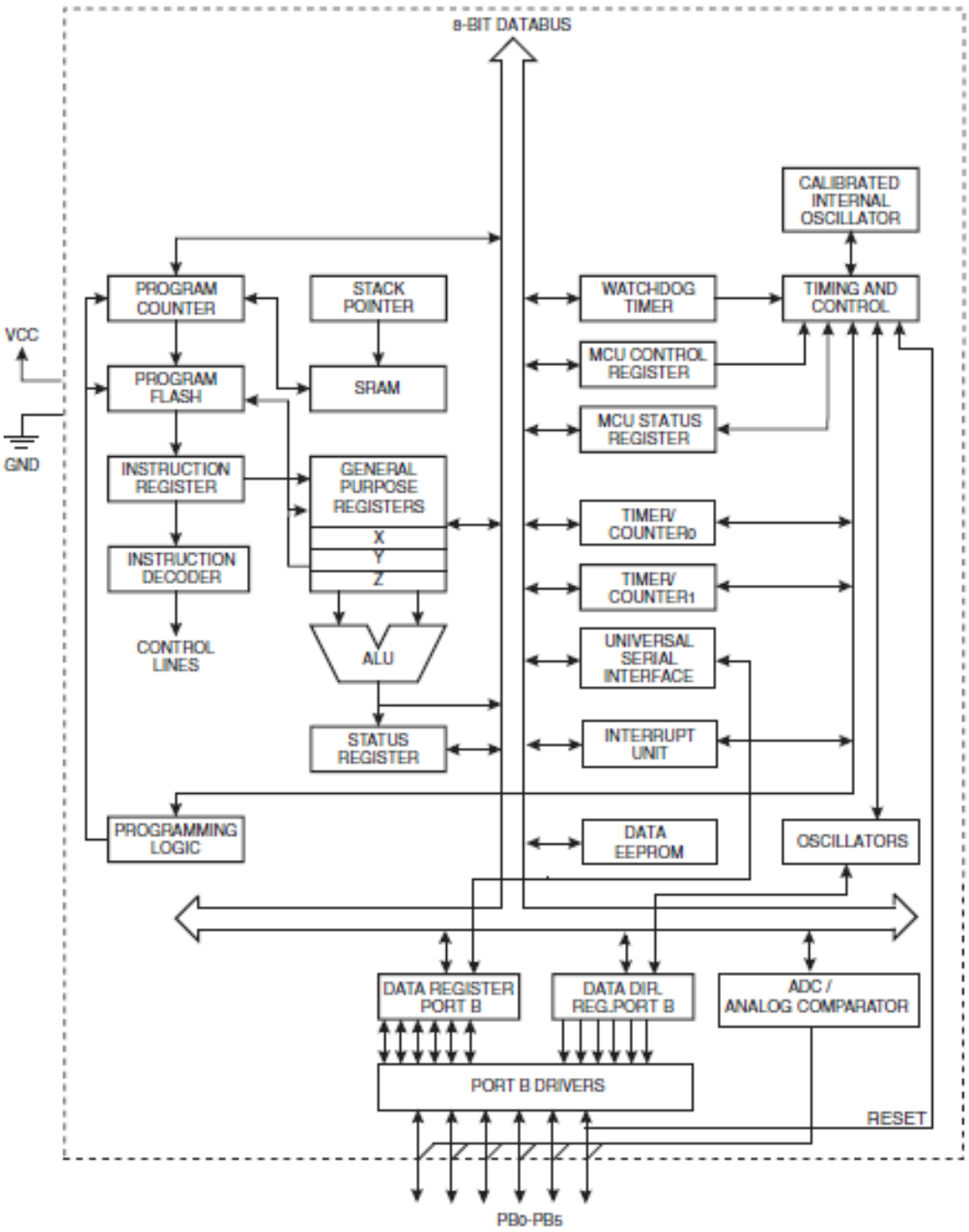
Simplified View of an AVR Microcontroller



- AVR is a family of microcontrollers originally developed by Atmel.
- Many popular AVR devices use an 8-bit RISC architecture.
- It combines processing, memory, and peripherals in one chip.
- It can interact with sensors and control external devices.
- The ATmega328P is a well-known AVR microcontroller used in the classic Arduino Uno.

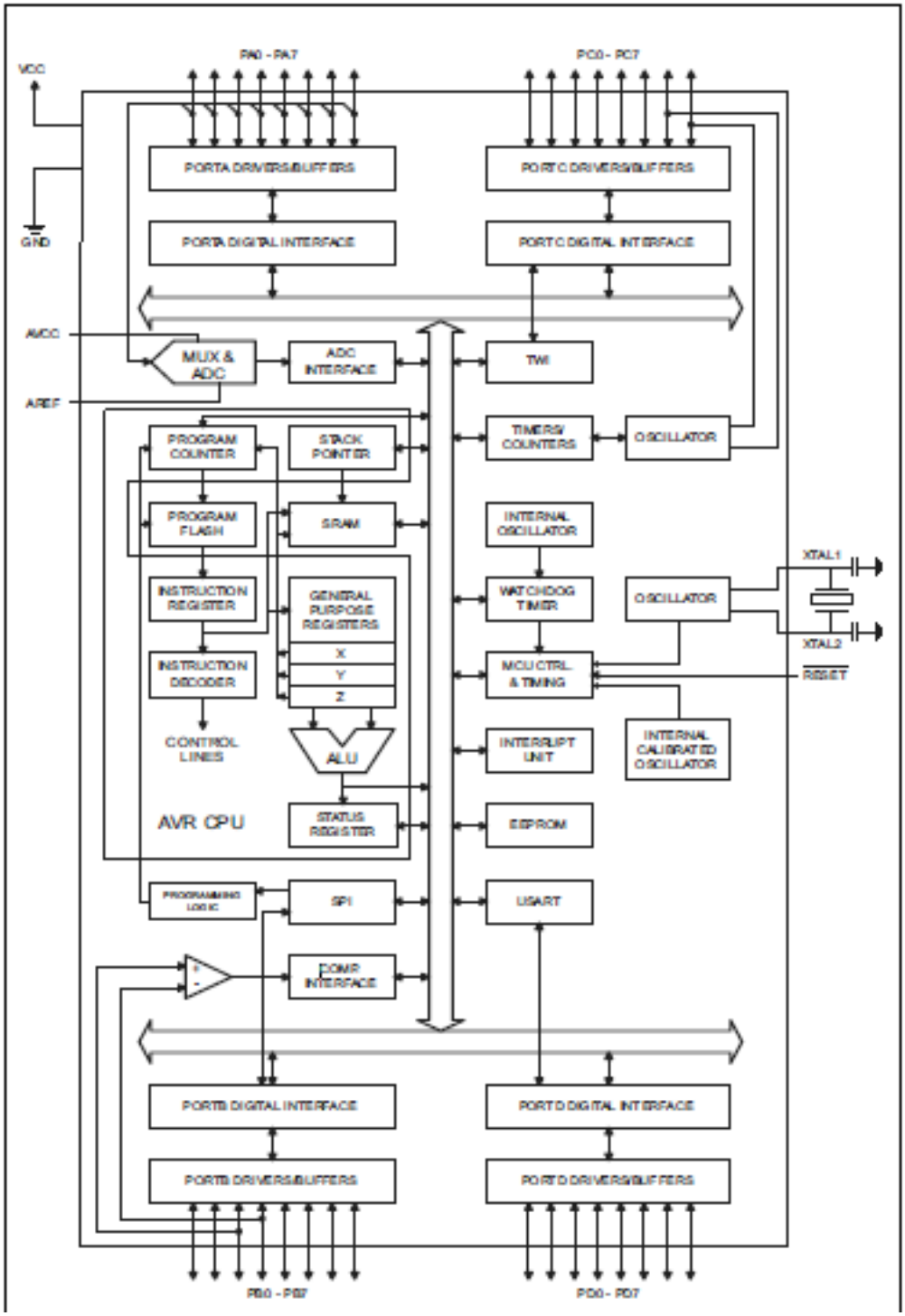
THE AVR MICROCONTROLLER

ATtiny25 Block Diagram



THE AVR MICROCONTROLLER

ATmega32 Block Diagram



8086 Microprocessor

- Intel introduced 8086 microprocessors in 1978.

This 16-bit microprocessor was a major improvement over the previous generation of microprocessors.

8086	8080/8085
1 megabyte (20-bit add. bus)	Memory of 64 kilobyte (16-bit add. bus)
16-bit Data bus	8-bit data bus
Pipelined processor (first single-chip *µpr.)*	Non-pipelined *µpr*

- In a system with **pipelining**, the data and the address bus are busy transferring data while the CPU is processing information.

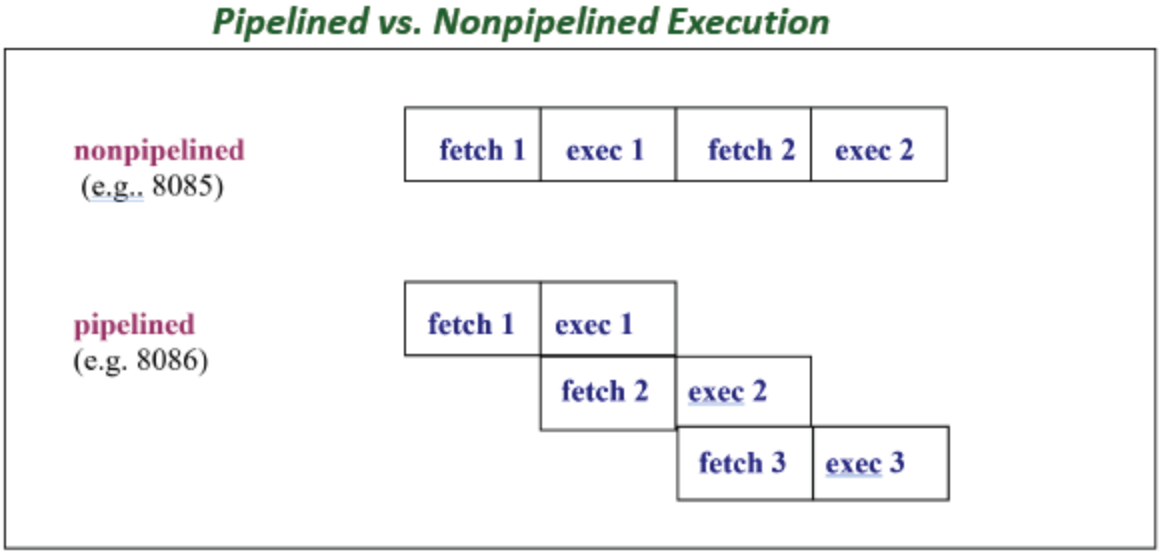
• *Pipelining*

❖ In 8085 microprocessors, the CPU could either ***fetch*** **or** ***execute*** at a given time.

CPU had to fetch an instruction from the memory, then execute it, then fetch again and execute it and so on.

❖ ***Pipelining*** is the simplest form to allow the CPU to ***fetch*** **and** ***execute*** at the ***same time***.

Note that the fetch and execute times can be different.



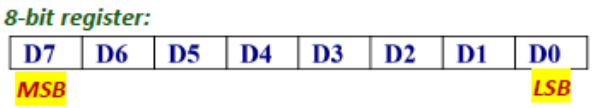
• *Registers of 8086*

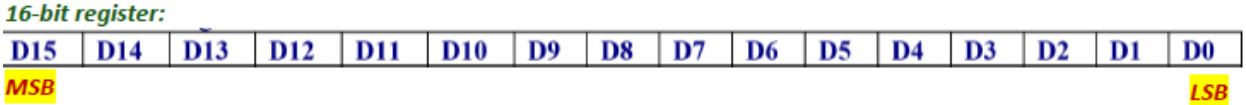
❖ In the CPU, registers are used ***store information temporarily***. The information can be ***one*** or ***two bytes*** of data, or the address of data.

❖ In 8088/8086 general-purpose registers can be accessed as either***16-bit*** or ***8-bit registers***.
All other registers can be accessed as full ***16-bit registers***.



The bits of the registers are numbered in descending order:





- ***Different registers*** are used for ***different functions***. Registers will be explained later within the context of instructions and their applications.
- The ***first letter*** of each general register indicates its use.
  - **AX** is used for the ***accumulator***.
  - **BX** is used for ***base addressing register.***
  - **CX** is used for ***counter loop operations***.
  - **DX** is used to point out ***data in I/O operations***.

- ***Registers of 8086***

Category	Bits	Register Names
General	16	**AX, BX, CX, DX**
	8	**AH, AL, BH, BL, CH, CL, DH, DL**
Pointer	16	**SP** (stack pointer), **BP**(base pointer)
Index	16	**SI** (source index), **DI**(destination index)
Segment	16	**CS** (code segment), **DS**(data segment) **SS** (stack segment), **ES**(extra segment)
Instruction	16	**IP** (instruction pointer)
Flag	16	**FR** (flag register)

General registers can be accessed as ***full 16 bits*** (such as AX), or as the ***high byte only (AH)*** or ***low byte only (AL).***
The others are NOT!!

Introduction to Assembly Language Programming

- ***Machine Language***

❖ Programs consist of 0s and 1s are called machine language.

❖ Assembly Language program consists of series of lines of Assembly language ***instructions***.

❖ ***Instructions*** consist of a ***mnemonic*** and ***operand(s).***

- ***MOV instruction***

MOV destination, source; *copy source operand to destination*

mnemonic *operands*

Example: (8-bit)

Comment sign

```
MOV     CL,55H    ;move 55H into register CL
MOV     DL,CL     ;move/copy the contents of CL into DL (now DL=CL=55H)
MOV     BH,DL     ;move/copy the contents of DL into BH (now DL=BH=55H)
MOV     AH,BH     ;move/copy the contents of BH into AH (now AH=BH=55H)
```

Example: (16-bit)

```
MOV     CX,468FH  ;move 468FH into CX (now CH =46 , CL=8F)
MOV     AX,CX     ;move/copy the contents of CX into AX (now AX=CX=468FH)
MOV     BX,AX     ;now BX=AX=468FH
MOV     DX,BX     ;now DX=BX=468FH
MOV     DI,AX     ;now DI=AX=468FH
MOV     SI,DI     ;now SI=DI=468FH
MOV     DS,SI     ;now DS=SI=468FH
MOV     BP,DS     ;now BP=DS=468FH
```

Rules regarding MOV instruction

❖ Data can be moved among all registers except the ***flag*** register. There are other ways to load the flag registers. To be studied later.

❖ Source and destination registers have to ***match in size***.

❖ Data can be moved among all registers (except flag reg.) but data can be moved ***directly*** into ***nonsegment*** registers only. You can't move data segment registers directly.

Examples:

```
MOV     BX,14AFH        ;move 14AFH into BX                    (legal)
MOV     SI,2345H        ;move 2345H into SI                    (legal)
MOV     DI,2233H        ;move 2233H into DI                    (legal)
MOV     CS,2A3FH        ;move 2A3FH into CS                    (illegal)
MOV     DS,CS           ;move the content of CS into DS        (legal)
MOV     FR,BX           ;move the content of BX into FR        (illegal)
MOV     DS,14AFH        ;move 14AFH into DS                    (illegal)
```

- **Important points**

  ❖ Data values cannot be loaded directly into (***CS***,***DS***,***SS*** and ***ES***)

```
MOV     AX,1234H        ; load 1234H into AX
MOV     SS,AX           ; load the value in AX into SS
```

  ❖ Sizes of the values:

```
MOV     BX,2H           ; BX=0002H,        BL:02H,  BH:00H

MOV     AL,123H         ; illegal (larger than 1 byte)
MOV     AX,3AFF21H      ; illegal (larger than 2 bytes)
```

• ADD *instruction*

ADD destination, source; *dest = dest + source*

mnemonic *operands*

Example:

```
MOV     AL,24H    ;move 24H into  AL
MOV     DL,11H    ;move 11H into  DL
ADD     AL,DL     ;AL=AL+DL      (AL=35H)(DL =11H)
```

```
MOV     CH,24H    ;move 24H into  CH
MOV     BL,11H    ;move 11H into  BL
ADD     CH,BL     ;CH=CH+BL      (CH=35H)
```

```
MOV     CH,24H    ;load one operand into CH
ADD     CH,11H    ;add the second operand to CH  (CH=35H)
```

ADD destination, source; *dest = dest + source*

mnemonic *operands*

- If destination register is followed by an immediate data as the source, it is called the immediate operand.

```
MOV     CH,24H
ADD     CH,11H
```

- 8-bit registers can hold FFH (255) as the maximum value. Addition of larger numbers can be performed by the 16-bit nonsegment registers.

```
MOV     AX,34EH
MOV     DX,6A5H
ADD     DX,AX          ;DX=DX+AX   (DX=9F3H)
MOV     CX,34EH
ADD     CX,6A5H        ;CX=34EH+6A5=9F3H
```

Introduction to Program Segments

• *Segment*

- A ***segment*** is an area of memory that includes up to 64K bytes and begins an address evenly divisible by 16 (such an address ends in 0H).
- Assembly Language Program consists of three segments:

❖ ***code segment***: contains the program code (instructions)
❖ ***data segment***: used to store data to be processed by the program
❖ ***stack segment:*** used to store information temporarily.

• *Logical and Physical Address*

- ***Physical Address*** is the 20-bit address that actually put on the address bus. (in 8086)

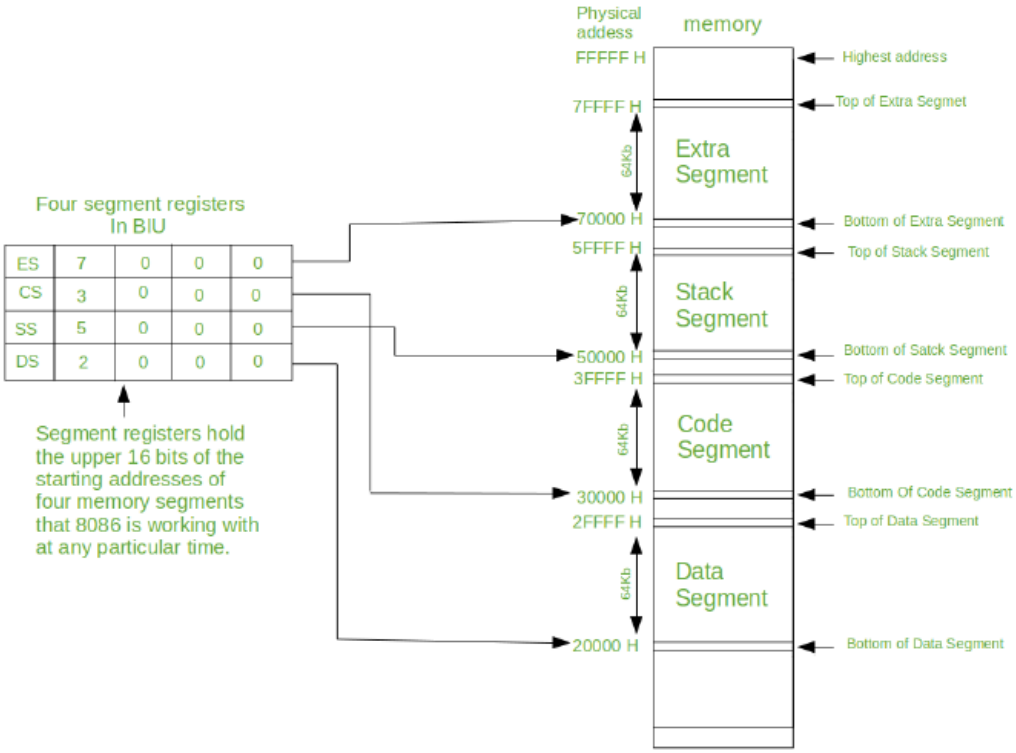
❖ Has a range of 00000H – FFFFFH

- ***Segment Address*** is a 16-bit address of the segment block. Each segment is a ***block*** of 64 KB of memory space.

- ***Offset Address*** is a location ***within*** 64K byte segment range. Has a range of 0000H – FFFFH

- ***Logical Address*** consists of ***segment address*** and ***offset address***.

- ***Memory Segmentation in 8086 Microprocessor***



- Logical Address: The address generated by the CPU in a program. In the 8086, it consists of a segment address and an offset.

- Physical Address: The actual address of the memory location where data or instructions are stored.

The logical address tells the processor which segment and position to use, while the physical address identifies the actual memory location.

• *Addressing in Code Segment*

- To execute a program, the 8086 fetches the instructions from the ***code segment***.
- The ***logical address*** of an instruction consists **CS** (Code Segment) and **IP**(instruction pointer).
- ***Logical Address*** **in Code segment** is represented by using **segment address in CS** register and **Offset Address in IP** register as follows:

CS:IP **(16 bit CS and 16 bit IP making total of 32 bits)**

Example: If CS register contains 2500H and IP register contains 95F3H. What is the ***Locical Adress*** in the code segment?

CS:IP → **2500:95F3** **(default in adressing is hex. You don't need H)**

• *Addressing in Code Segment*

Physical Address is generated by shifting the CS one hex digit to the left and adding IP. Physical address is ***20 bit address*** which can be generated by using a logical address as follows.

1. **Start with CS**
2. **Shift left CS (insert 0 as the Least significant digit)**
3. **Add IP**

Example: If CS register contains 1980H and IP register contains 78FEH. What is the ***Physical Adress*** in the code segment?

Logical address: CS:IP → **1980:78FE**

1.	**Start with CS**	1980	
2.	**Shift left CS**	19800	
3.	**Add IP**	78FE	(19800+ 78FE =210FE)

Physical address: The microprocessor will retrieve the instruction from the memory locations starting from **210FE** ***(20 bit address).***

• *Addressing in Code Segment*

Example: If CS=24F6H and IP=634AH, determine:

a) The logical address
b) The offset address
c) The physical address
d) The lower range of the code segment
e) The upper range of the code segment

Solution:

a) The logical address is; **24F6:634A**
b) The offset address is; **634A**
c) The Physical address is; **24F60+634A= 2B2AA**
d) The lower range of the code segment: **24F6:0000 → 24F60+0000 = 24F60**
e) The upper range of the code segment: **24F6:FFFF → 24F60+FFFF = 34F5F**

• *Addressing in Data Segment*

- The area of memory allocated strictly for data is called ***data segment.***
- ***Data segment*** contains ***variables*** containing single values and arrays of values, where ***code segment*** only contain program ***instructions***.
- ***Logical Address*** **in Data Segment** is represented by using **segment address in DS** register and **Offset Address in BX, SI or DI** registers.

DS:BX
DS:SI
DS:DI

- At any time ***three locations*** in the data segment are pointed with ***DS:BX***, ***DS:SI*** and ***DS:DI*** respectively.

- ***Addressing in Data Segment***

Example: If DS=7FA2H and the offset is 438EH, determine:

a) The physical address
b) The lower range of the data segment
c) The upper range of the data segment
d) Show the logical address

Solution:

a) The Physical address is; **7FA20+438E = 83DAE**
b) The lower range: **7FA20+0000= 7FA20**
c) The upper range: **7FA20+FFFF = 8FA1F**
d) The logical address is; **7FA2:438E**

- ***Addressing in Data Segment***

Why do we use data segment?

Assume that a program is needed to add 5 bytes of data (25H, 12H, 15H,1FH and 2BH)

One way:

```
MOV  AL,00H      ;initialize AL
ADD  AL,25H
ADD AL,12H
ADD AL,15H
ADD AL,1FH
ADD AL,2BH       ; AL=25+12+15+1F+2B
```

code and data are mixed
(bad programming practice)

Better way: Assume that the Data segment contains the array of bytes starting from **offset address 0200H**.

```
MOV  AL,0 ;clear AL
ADD AL,[0200];add the contents of DS:200 to AL
ADD AL,[0201];add the contents of DS:201 to AL
ADD AL,[0202];add the contents of DS:202 to AL
ADD AL,[0203];add the contents of DS:203 to AL
ADD AL,[0204];add the contents of DS:204 to AL
```

code and data are separated
(good programming practice)

DS:01FF	?
DS:0200	25
DS:0201	12
DS:0202	15
DS:0203	1F
DS:0204	2B
DS:0205	?

Data Segment

- ***Little endian convention***

Given 8-bit (1-byte) data, bytes are stored one after the other in the memory. However given16-bit (2-bytes) of data how are date stored?

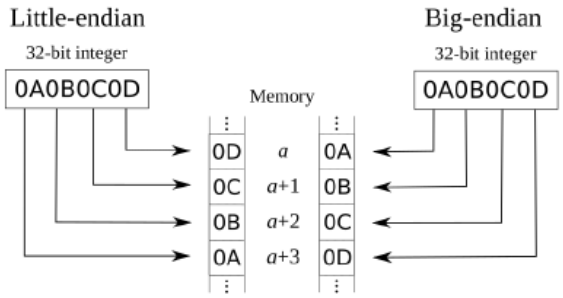
Example:
```
MOV AX,35F3H    ;load 35F3H into AX
MOV [1500],AX   ; copy contents of AX to offset 1500H
```

In such a case the low byte goes to the low memory location and high byte goes to the high memory location.

DS:1500 = F3 DS:1501 = 35

This convention is called ***little endian convention***: This convention is used by ***Intel***.

Big endian convention is the opposite, where the high byte goes to the low address and low byte goes to the high address. ***Motorolla*** microprocessor uses this convention.



Addressing in Stack Segment

- ***What is a stack, and why is it needed?***

The stack is a section of RAM used by the CPU to store information temporarily. CPU needs this storage area since there are only limited number of registers.

- ***How stacks are accessed***

SS (stack segment) and ***SP*** (stack pointer) must be loaded to access stack in the memory.

Every register in the CPU (except segment registers and SP) can be stored in the stack and loaded from the stack.

- ***Logical Address*** **in Stack Segment** is represented by using **segment address in SS r**egister and **Offset Address in SP** register.

SS:SP

- ## Stack: Pushing and Popping Operations

- ***Push Instruction***

  Storing the CPU register in the stack is called a ***push***.

PUSH source; — *Copy the content of source (16-bit register) into stack*

mnemonic operand — * SP register is decremented by 2

Example: Given that SP=1456H, what are the contents of AX, top of the stack and SP after the execution of the following instruction.

```
MOV  AX,2174H
PUSH AX
```

Solution: AX=2174H (stays the same), SP=1454H (decremented by 2),

SS:1453	?
SS:1454	74
SS:1455	21
SS:1456	?
SS:1457	?
SS:1458	?

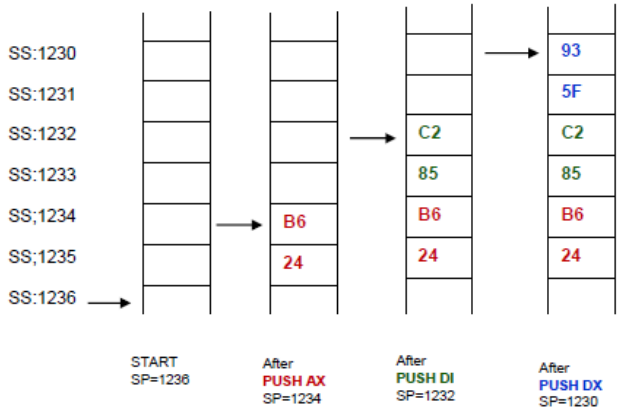
Stack Segment

- ***Pushing onto the stack***

  Storing the CPU register in the stack is called a ***push***.

Example: SP=1236H, AX=24B6H, DI=85C2H, and DX=5F93H, show the contents of the stack as each of the following instructions is executed.

```
PUSH  AX
PUSH  DI
PUSH  DX
```



Solution:
Note that in 80x86 the lower byte of the register is stored to the lower address. **(Little Endian Convention)**

- ***Pop Instruction***

  Loading the contents of the stack into the CPU register is called a ***pop***.

POP destination; ***Copy the top of the stack into destination (16-bit register)***

mnemonic *operand* * SP register is incremented by 2

Example: Assume that SP=134AH and the illustration on the right shows the content of the top of the stack. What will be the content of AX and SP after of after the execution of the following instruction.

POP AX

Solution: AX=FC76H and SP=134CH (incremented by 2),

SS:1349	?
SS:134A	76
SS:134B	FC
SS:134C	?
SS:134E	?
SS:134F	?

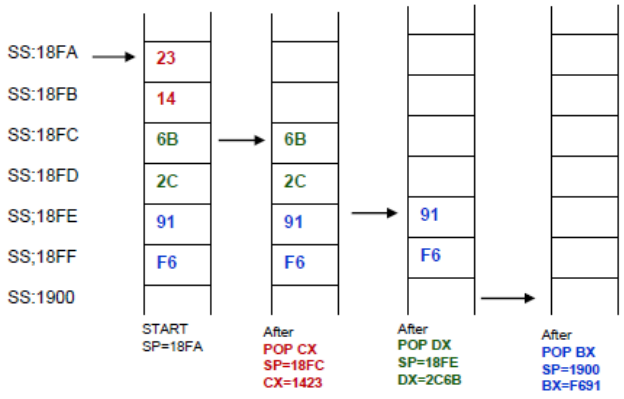
Stack Segment

- ***Popping the stack***

  Loading the contents of the stack into the CPU register is called a ***pop***.

  Example: Assume that the stack is shown below, and SP=18FAH, show the contents of the stack and registers as each of the following instructions is executed.

POP CX
POP DX
POP BX



Solution:
Note that in 80x86 the byte in the Low address goes into the low byte. the byte in the high address goes into the high byte. **(Little Endian Convention)**

- ***Logical vs. physical address of the stack***

- Calculating the physical address for the stack, the same principle is applied as was used for the code and data segments. Physical address depends on the value of stack segment (SS) register and the stack pointer (SP).

Example: If SS=3500H and SP:FFFEH

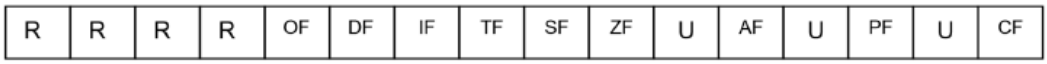
a)	Calculate the physical address:	35000+FFFE = 44FFE
b)	Calculate the lower range of the stack:	35000+0000 = 35000
c)	Calculate the upper range of the stack segment:	35000+FFFF = 44FFF
d)	Show the logical address of the stack:	3500:FFFE

- **The Flag Register (FR) and bit fields**

  - The **flag register** is a **16-bit register** sometimes referred as the ***status*** register. Although the register is 16-bit. Not all the bits are used.

  - **Conditional flags:** 6 of the flags are called the conditional flags, meaning that they indicate some condition that resulted after an instruction was executed. These 6 are: **CF, PF, AF, ZF, SF,** and **OF**.

  - The 16 bits of the flag registers:



R	R	R	R	OF	DF	IF	TF	SF	ZF	U	AF	U	PF	U	CF

R=	reserved	SF=	sign flag
U=	undefined	ZF=	zero flag
OF=	overflow flag	AF=	auxiliary carry flag
DF=	direction flag	PF=	parity flag
IF=	interrupt flag	CF=	carry flag
TF=	trap flag		

The Flag Register (FR) and bit fields

CF, the Carry Flag: This flag is set whenever there is a carry out, either from d7 after an 8-bit operation, or from d15 after a 16-bit data operation.
PF, the Parity Flag: After certain operations, the parity of the result's low-order byte is checked. If the byte has an even number of 1s, the parity flag is set to 1; otherwise, it is cleared.
AF, the Auxiliary Carry Flag: If there is a carry from d3 to d4 of an operation this bit is set to 1, otherwise cleared (set to 0).
ZF, the Zero Flag: The ZF is set to 1 if the result of the arithmetic or logical operation is zero, otherwise, it is cleared (set to 0).
SF, the Sign Flag: MSB is used as the sign bit of the binary representation of the signed numbers. After arithmetic or logical operations, the MSB is copied into SF to indicate the sign of the result.
TF, the Trap Flag: When this flag is set it allows the program to single step, meaning to execute one instruction at a time. Used for debugging purposes.
IF, Interrupt Enable Flag: This bit is set or cleared to enable or disable only the external interrupt requests.
DF, the Direction Flag: This bit is used to control the direction of the string operations.
OF, the Overflow Flag: This flag is set whenever the result of a signed number operation is too large, causing the high-order bit to overflow into the sign bit.

- ***Manipulating the Flag Register***
- There are two instructions that can be used to use/change the content of the flag register.
- **PUSHF Instruction**

```
PUSHF          ;   Copy the flag register into stack
                   * SP register is decremented by 2
```

Example: Copy the content of the flag register into register AX.

```
PUSHF     ; copy the content of Flag register into the stack.
POP AX    ; copy from the stack into AX
```

- ***Manipulating the Flag Register***

- There are two instructions that can be used to use/change the content of the flag register.
- **POPF Instruction**

POPF ; ***Copy from the stack into the flag register***

* SP register is incremented by 2

Example: Clear the flag bits. (Make the flag bits to be all 0)

```
MOV AX,0000H
PUSH AX  ; now top of the stack contains 16-bit 0.
POPF ; copy the content of stack into flag register.
```